Reviewer Pack — Minimal Order of Groupoid Categorical Dimensions and Communi…

Entry d975d59a0781 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 07:39:01 UTC

Minimal Order of Groupoid Categorical Dimensions and Communication Complexity Rank Inequality

Entry ID: `d975d59a0781` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 07:39:01 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Groupoids) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every d-regular graph G, the minimal categorical dimension of its associated groupoidal category K(G) is linearly correlated with its communication complexity rank r(K(G)), such that min_dim(K(G)) = Ω(r(K(G))).

Rationale (proposer's reasoning):

> Groupoids provide a categorical framework for studying symmetries and transformations in complex systems. Their categorical dimensions capture the complexity of these transformations, which could be related to communication complexity ranks, providing insights into the difficulty of distributing information efficiently.

Taxonomy category: `groupoid_categorization` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c85189261429ca3c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all d-regular graphs G with n ≤ 40 vertices, min_dim(K(G)) meets the inequality min_dim(K(G)) ≥ c * r(K(G)) for a constant c > 0 and at least 80% of seeds produce this relationship.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "categorical dimension" AND "groupoid" AND "communication complexity" AND "matrix rank" - "minimal order groupoid" AND "dimension" AND "communication complexity" AND "rank inequality" - "category theory" AND "groupoid categorical dimensions" AND "communication complexity rank" AND ">="

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0:
            return None
        graph = {i: [] for i in range(n)}
        edges_added = set()
        for _ in range(d * n // 2):
            while True:
                u = random.randint(0, n-1)
                v = random.randint(0, n-1)
                if u != v and (u, v) not in edges_added and (v, u) not in edges_added:
                    graph[u].append(v)
                    graph[v].append(u)
                    edges_added.add((u, v))
                    break
        return graph

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                return None
```

```
        for j in range(n):
            A[i][j] /= A[i][i]
        for j in range(m):
            if j != i and A[j][i] != 0:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
    return A

def matrix_rank(A):
    rank = 0
    A = gaussian_elimination(A)
    if A is None:
        return 0
    for row in A:
        if any(row):
            rank += 1
    return rank

def min_categorical_dimension(graph):
    n = len(graph)
    adjacency_matrix = [[int(v in graph[u]) for v in range(n)] for u in range(n)]
    rank = matrix_rank(adjacency_matrix)
    return rank

n_max = 40
instances_tested = 0
total_min_dim = 0
total_comm_complexity = 0

for n in [5, 10, 15, 20, 30, 40]:
    d = random.randint(2, min(n-1, 5))
    graph = generate_d_regular_graph(n, d)
    if graph is None:
        continue
    instances_tested += n * (n - 1) // 2
    min_dim = min_categorical_dimension(graph)
    comm_complexity = matrix_rank([graph[u] for u in range(n)])
    total_min_dim += min_dim
    total_comm_complexity += comm_complexity

if instances_tested < 30:
    return {
        "metric_name": "min_dim vs comm_complexity",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

mean_min_dim = total_min_dim / instances_tested
mean_comm_complexity = total_comm_complexity / instances_tested

if mean_min_dim < 0.1 * mean_comm_complexity:
    return {
```

```
            "metric_name": "min_dim vs comm_complexity",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "inequality_violated"
        }

    return {
        "metric_name": "min_dim vs comm_complexity",
        "metric_value": mean_min_dim / mean_comm_complexity,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"inequality_violated\" first_failing_seed={first
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_571d355f.py", line 126, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_571d355f.py", line 84, in run_trial
    comm_complexity = matrix_rank([graph[u] for u in range(n)])
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_571d355f.py", line 58, in matrix_rank
    A = gaussian_elimination(A)
        ^^^^^^^^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_571d355f.py", line 53, in gaussian_elim
    A[j][k] -= factor * A[i][k]
    ~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with a different seed or ensure that error handling is in place to collect results even if an exception occurs.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13484
2	preregistration	ollama_remote	glm4:latest	0	0	9329
3	novelty	ollama_remote	glm4:latest	0	0	8931
4	novelty	ollama_remote	glm4:latest	0	0	9041
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29290
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10936
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11989
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13432
9	judge	ollama_remote	glm4:latest	0	0	18190

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 124623 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d975d59a0781.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d975d59a0781.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d975d59a0781.tar.gz` (if generated)